

并行计算 习题二 空白练习卷

来源：习题/ipp-source-main/ipp-source-main/homework/习题二.pdf

说明：本练习卷已删除原 PDF 中填好的答案；选择题、填空题和答题区均留空。

一、选择题

1. 高性能计算的性能衡量单位是 flops，那么 1 Tflops 是（ ）。
 - A. 每秒千万亿次的浮点运算
 - B. 每秒万亿次的浮点运算
 - C. 每秒百亿亿次的浮点运算
 - D. 每秒拾亿次的浮点运算
2. 下列哪台高性能计算机是目前性能最高的超级计算机（ ）。
 - A. 天河二号
 - B. Sunway TaihuLight
 - C. Summit
 - D. Sierra
3. 下列计算机中哪个不属于分布式内存系统的计算机（ ）。
 - A. 集群
 - B. 超级计算机
 - C. 冯诺依曼计算机
 - D. 数据中心
4. 下列关于 MPI 函数库中 MPI_Send() 与 MPI_Recv() 说法正确的是（ ）。
 - A. MPI 中消息发送函数 MPI_Send() 总是阻塞的
 - B. MPI 中消息接收函数 MPI_Recv() 总是阻塞的
 - C. MPI 中消息发送函数 MPI_Send() 总是缓冲的
 - D. MPI 中消息接收函数 MPI_Recv() 总是缓冲的
5. 下列关于 MPI 中函数或变量名不正确的是（ ）。
 - A. MPI_SUCCESS
 - B. MPI_Init

C. MPI_Comm_world

D. MPI_Comm_rank

6. MPI 中接收函数 MPI_Recv() 在不知道以下哪个信息的情况下不能接收到消息 ()。

A. 消息中的数据量

B. 消息中发送者

C. 消息的数据类型

D. 消息的标签

7. 在 Pthreads 中, 下列哪一个不是用于保护临界区的方式 ()。

A. 锁机制

B. 忙等待

C. 信号量

D. 互斥量

8. 在 OpenMP 中, 假设利用 2 个线程对具有 5 次迭代的 for 循环进行并行, 下列哪种调度方式可能使线程 1 执行迭代 0,1,4; 线程 2 执行迭代 2,3 ()。

A. schedule(static,3)

B. schedule(static,1)

C. schedule(static,2)

D. schedule(dynamic,1)

9. 在 MPI 中, 如果要将一个具有 100 个分量的数组分发给通信子中的所有进程 (10 个进程), 能够采用哪个集合通信函数 ()。

A. MPI_Reduce(...)

B. MPI_Scatter(...)

C. MPI_Send(...)

D. MPI_Bcast(...)

10. 在 OpenMP 中, 如果要进行显式路障设置, 应采用哪个指令 ()。

A. reduction

B. barrier

C. critical

D. atomic

二、填空题

1. 常用的 Linpack 基准测试程序有 _____ 和 _____。
2. MPI 的英文全称为 _____；集合通信的英文为 _____。
3. MPI 的通信分为 _____ 和 _____。
4. 请写出在 3 个主机上启动 MPD 进程，形成 MPI 运行时环境的命令：
_____。（目标主机列表由 \$HOME/mpd.hosts 文件指定）
5. 只有接收进程可以使用通配符参数，常见的通配符参数包括 _____ 和 _____。
6. OpenMP 适用于 _____ 系统进行并行程序设计，其中 #pragma omp critical 的功能是：
_____。
7. OpenMP 中，一条编译指导语句由 _____ 和 _____ 组成。
8. 在 OpenMP 中，并行 for 循环的指令包括：for 指令和 _____。
9. MPI_Scatter(a, local_n, MPI_DOUBLE, local_a, local_n, MPI_DOUBLE, 0, comm) 这条语句执行的功能是：
_____。
10. #pragma omp parallel num_threads(thread_count) reduction(+:global_result) 这条编译指导语句的作用是：
_____。

三、简答题

1. 在 Linux 系统中，对文件名为 hello.c 的 MPI 源程序、Pthreads 源程序和 OpenMP 源程序，分别写出采用 3 个进程或线程的编译和运行命令。

答：

2. 考虑下面的循环。该循环存在循环依赖，请找一种方法消除循环依赖，并且并行化这个循环。

```
a[0] = 0;
for (i = 1; i < n; i++)
    a[i] = a[i-1] + 2*i;
```

答：

3. 下述代码是非递归深度优先搜索的代码片段，并假设 `Feasible(curr_tour, nbr)` 始终为真。现考虑 4 个城市的 TSP 问题，请画出栈和当前回路的运行状态，直到 `curr_tour` 为 _____；请找出下述并行代码的临界区，并采用 OpenMP 的方式解决临界区问题。

```
Push_copy(stack, tour); // 将回路压入栈中
while (!Empty(stack)) {
    curr_tour = Pop(stack); // 将回路弹出栈中
    if (City_count(curr_tour) == n) {
        if (Best_tour(curr_tour))
            Update_best_tour(curr_tour);
    } else {
        for (nbr = n-1; nbr >= 1; nbr--)
            if (Feasible(curr_tour, nbr)) {
                Add_city(curr_tour, nbr);
                Push_copy(stack, curr_tour, avail);
                Remove_last_city(curr_tour);
            }
    }
    Free_tour(curr_tour);
}
```

答:

4. 判断下列程序是否能够安全通信，并说明原因。若不安全，请给出相应解决方案。

```
comm = MPI_COMM_WORLD;
MPI_Comm_rank(MPI_COMM_WORLD, &rank);

if (rank == 0) {
    MPI_Recv(x2, 3, MPI_INT, 1, tag, comm, &status); // A 语句
    MPI_Send(x1, 3, MPI_INT, 1, tag, comm);           // B 语句
}

if (rank == 1) {
    MPI_Recv(x1, 3, MPI_INT, 0, tag, comm, &status); // C 语句
    MPI_Send(x2, 3, MPI_INT, 0, tag, comm);           // D 语句
}
```

答：

5. 请按自己的理解说明并行计算和分布式计算的异同点。

答：

四、程序阅读题

1. 写出下述程序的运行结果。

```
int k = 100;
#pragma omp parallel for private(k)
for (k = 0; k < 2; k++) {
    printf("k=%d\n", k);
}
printf("last k=%d\n", k);
```

答:

2. 写出以下程序段的运行结果；若删除 `#pragma omp parallel num_threads(2){}`，程序段运行结果又如何？

```
int j = 0;

#pragma omp parallel num_threads(2)
{
    #pragma omp for
    for (j = 0; j < 2; j++)
        printf("j=%d, ThreadId=%d\n", j, omp_get_thread_num());
}
```

答:

五、程序题

1. 请将以下程序改写为 10 个线程执行的 OpenMP 并程序，以使用莱布尼茨级数估计 pi。

```
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char* argv[]) {
    long long n, i;
    double factor;
    double sum = 0.0;
    n = 1000;

    for (i = 0; i < n; i++) {
        sum += factor / (2*i + 1);
        factor = -factor;
    }

    sum = 4.0 * sum;
    printf("Our estimate of pi = %.14f\n", sum);
    return 0;
}
```

答：

2. 采用 MPI 实现梯形积分。要求：(1) 主函数采用点对点通信，即 0 号进程接收消息并打印，其它进程发送消息；(2) 接收用户输入函数 Get_input 采用 MPI_Bcast 实现。已给 Trap 函数如下。

```
double Trap(
    double left_endpt /* in */,
    double right_endpt /* in */,
    int trap_count /* in */,
    double base_len /* in */) {
    double estimate, x;
    int i;
    estimate = (f(left_endpt) + f(right_endpt)) / 2.0;
```

```
for (i = 1; i <= trap_count-1; i++) {  
    x = left_endpt + i * base_len;  
    estimate += f(x);  
}  
estimate = estimate * base_len;  
return estimate;  
} /* Trap */
```

答:
